

Expense Tracker MVP

Backend & Frontend Task Document (MERN + Next.js)

Stack: MongoDB | Express | React (Next.js) | Node.js

Date: July 16, 2026

Goal: Build a working MVP where a user can register, log in, and manage personal expenses (create, read, update, delete), with a simple monthly summary by category.

1. Architecture

Next.js (Frontend) --> REST JSON --> Express (Node.js) --> MongoDB

- Frontend: Next.js App Router + JWT stored client-side
- Backend: Express API with JWT auth middleware
- Database: MongoDB with Mongoose models (User, Expense)

2. MVP Scope

In scope

- Auth (register / login / me)
- Expense CRUD
- Filters (date range, category)
- Monthly summary (total + by category)

Out of scope

- Budgets & alerts
- Recurring expenses
- Multi-currency FX
- Receipt upload / OCR
- Shared household accounts

3. Suggested milestones

- Week 1 - Backend setup + auth + expense CRUD (API usable in Postman)
- Week 2 - Frontend auth + expense list/forms (end-to-end CRUD)
- Week 3 - Summary UI + polish + README (demo-ready MVP)

4. Backend Tasks

Stack: Node.js | Express | MongoDB | Mongoose | JWT | API base: /api

Phase 0 - Project setup

B0.1 | Initialize Express server with config/models/routes/controllers/middleware

Done when: npm run dev starts on PORT

B0.2 | Add deps: express, mongoose, dotenv, cors, bcryptjs, jsonwebtoken, express-validator

Done when: package.json lists all deps

B0.3 | Create .env.example (PORT, MONGODB_URI, JWT_SECRET, CLIENT_URL)

Done when: Documented and copyable

B0.4 | Connect MongoDB via Mongoose with startup logs

Done when: App exits cleanly if DB fails

B0.5 | Enable CORS for CLIENT_URL and JSON body parsing

Done when: Frontend can call API without CORS errors

Phase 1 - Auth

B1.1 | User model: name, email (unique), passwordHash, timestamps

Done when: Schema validates required fields

B1.2 | POST /api/auth/register - hash password, create user, return JWT

Done when: Duplicate email = 409; no password in response

B1.3 | POST /api/auth/login - verify credentials, return JWT

Done when: Wrong credentials = 401

B1.4 | GET /api/auth/me - current user from JWT

Done when: Missing/invalid token = 401

B1.5 | Auth middleware: Bearer token -> req.user

Done when: Protects expense routes

Phase 2 - Expenses CRUD

B2.1 | Expense model: userId, title, amount, category, date, note

Done when: userId indexed; amount > 0

B2.2 | POST /api/expenses - create for logged-in user

Done when: Always scoped to req.user.id

B2.3 | GET /api/expenses - filters: from, to, category, page, limit

Done when: Newest first; pagination meta

B2.4 | GET /api/expenses/:id - owned expense only

Done when: Other user's expense = 404

B2.5 | PUT /api/expenses/:id - update owned expense

Done when: Partial updates allowed

B2.6 | DELETE /api/expenses/:id - delete owned expense

Done when: 204 or success message

Phase 3 - Summary & validation

B3.1 | GET /api/expenses/summary - total + by-category

Done when: MongoDB aggregation

B3.2 | Input validation on all write endpoints

Done when: Invalid body = 400 with field errors

B3.3 | Central error handler middleware

Done when: Consistent JSON error shape

B3.4 | Seed script for demo user + sample expenses

Done when: npm run seed works

Phase 4 - Hardening (MVP+)

B4.1 | Rate-limit auth routes

Done when: Brute-force login limited

B4.2 | Helmet + security headers

Done when: Headers present

B4.3 | GET /api/health health check

Done when: Returns { status: "ok" }

B4.4 | README for running the API

Done when: Another developer can run in <5 minutes

Backend definition of done

- All auth + expense endpoints work via Postman/Thunder Client
- Every expense query is filtered by userId
- Passwords never returned in responses
- .env is not committed; .env.example is

5. Frontend Tasks

Stack: Next.js (App Router) | React | Fetch/Axios | NEXT_PUBLIC_API_URL

Phase 0 - Project setup

F0.1 | Initialize Next.js app in client/ (App Router, TypeScript)

Done when: npm run dev opens home page

F0.2 | Configure .env.local with NEXT_PUBLIC_API_URL

Done when: API base URL injectable

F0.3 | Folders: `app/`, `components/`, `lib/`, `hooks/`, `types/`

Done when: Clear separation of concerns

F0.4 | API client (`lib/api.ts`) with auth header injection

Done when: Central place for all HTTP calls

F0.5 | Shared types: `User`, `Expense`, `Summary`

Done when: Types match backend responses

Phase 1 - Auth UI & session

F1.1 | Register page (`/register`)

Done when: Client validation + API error display

F1.2 | Login page (`/login`)

Done when: Success stores JWT

F1.3 | Auth context/provider

Done when: App knows logged-in user globally

F1.4 | Protect dashboard routes

Done when: Guests redirected to `/login`

F1.5 | Navbar with user name + Logout

Done when: Logout clears token and redirects

Phase 2 - Expense management UI

F2.1 | Dashboard layout (`/dashboard`)

Done when: Summary + recent expenses

F2.2 | Expense list (`table/cards`)

Done when: Empty state when no data

F2.3 | Add expense form

Done when: POST `/expenses` then refresh list

F2.4 | Edit expense form

Done when: Pre-fills; PUT `/expenses/:id`

F2.5 | Delete with confirm dialog

Done when: DELETE then remove from UI

F2.6 | Filters: `date range` + `category` + `title search`

Done when: Query params sent to API

Phase 3 - Summary & polish

F3.1 | Monthly summary cards (`total`, `count`)

Done when: Uses `/expenses/summary`

F3.2 | Category breakdown (`bars` or `list`)

Done when: Totals per category

F3.3 | Loading skeletons + toast/errors

Done when: Clear UX on slow/failed requests

F3.4 | Responsive layout (mobile + desktop)

Done when: Usable on phone-width

F3.5 | Currency formatting

Done when: Amounts readable and consistent

Phase 4 - Frontend hardening (MVP+)

F4.1 | Form validation with helpful messages

Done when: Invalid fields blocked before submit

F4.2 | Disable buttons while submitting

Done when: No double-submit

F4.3 | 401 interceptor - auto logout / redirect

Done when: Expired tokens handled

F4.4 | README for running the client

Done when: Another developer can run in <5 minutes

Suggested routes

- / - Landing / redirect to dashboard
- /login - Login
- /register - Register
- /dashboard - Summary + recent expenses
- /expenses - Full list + filters
- /expenses/new or modal - Add expense
- /expenses/[id]/edit or modal - Edit expense

Frontend definition of done

- User can register, login, logout
- User can add, edit, delete, filter expenses
- Dashboard shows monthly total and category breakdown
- All API errors surface as readable messages
- App works on desktop and mobile viewport

6. Quick reference

User model

- _id, name, email (unique), passwordHash, createdAt, updatedAt

Expense model

- _id, userId, title, amount, category, date, note, createdAt, updatedAt

Categories

- Food, Transport, Rent, Utilities, Entertainment, Shopping, Health, Other

Key API endpoints

- POST /api/auth/register | POST /api/auth/login | GET /api/auth/me
- GET/POST /api/expenses | GET/PUT/DELETE /api/expenses/:id
- GET /api/expenses/summary

Environment variables

- Server: PORT, MONGODB_URI, JWT_SECRET, CLIENT_URL
- Client: NEXT_PUBLIC_API_URL

7. Checklist summary

Use BACKEND_TASKS.md and FRONTEND_TASKS.md in this docs folder as living checklists while building. Mark each ID (B0.1, F1.2, ...) done as you complete it.